Every Typed Song Ends: Strong Normalization for the Two-Move Hilbert Coin

Working paper v0.1, Project secret3, August 11, 2026

Abstract

This paper closes line L1. The preservation paper showed that a typed term keeps its proposition through every computation step; this one shows that the computation cannot go on forever. The theorem strongNorm establishes strong normalization for the coin: every term typed by the Hilbert system over K and S is strongly normalizing, no reduction sequence from it is infinite, along any strategy, quantified over all of them. The proof is the reducibility method, Tait's idea in Girard's dress, instantiated for combinatory logic, where it enjoys a famous mercy: there are no binders, so there is no substitution lemma, and the whole development is closed terms all the way down. Strong normalization is accessibility under the step relation; reducibility is defined by recursion on types with normalization carried at every arrow; the three closure conditions are compiled, forward closure, and backward closure for neutral terms with an inner descent on the argument's own normalization; neutrality is defined semantically against a five-line inventory of the calculus's values, which makes the key inversion, steps of a neutral-headed application happen in the parts, a four-clause lemma. The axioms are then made computable by well-founded descent on their cargos' normalization, the main induction over derivations is three lines, and the theorem falls out as the first projection. Read together with stage two of the reader campaign, the paper's corollary in prose: the verified engine, pointed at any typed program, halts, and halts at a normal form.

1. The debt and the method

The book's garden section says: every typed song ends. Until this paper, that sentence leaned on Tait (1967) as literature; the project's discipline is that sentences lean on theorems it owns. The method is the classical one. Termination cannot be proved by induction on terms, because reduction can grow a term before shrinking it; the repair is to strengthen the induction hypothesis into a type-indexed predicate, reducibility, strong enough to propagate through application and to imply normalization. Girard's presentation isolates what the predicate must satisfy as three conditions; this paper compiles them for the coin, and reports the mechanization choices, since two of them extend lessons already logged in this series.

2. The definitions

Strong normalization is accessibility: a term is SNorm when every one-step reduct is, the empty condition holding at normal forms.

```
data SNorm : Term -> Type where
  MkSN : ((u : Term) -> Step t u -> SNorm u) -> SNorm t
```

Values are the calculus's under-applied heads, five shapes: K, S, K x, S x, S x y. Neutral means not a value. Two small lemmas carry all the structure the proof needs. First, rootNeeds: a step out of an application with a neutral head is a step in the head or a step in the argument, never a root firing, four clauses, the value inventory doing the refutations. Second, neuApp: a neutral head keeps its application neutral, three clauses. Reducibility is then defined by recursion on the type, with normalization carried at every arrow, which settles at the outset the question that blocks the naive de-

velopment for closed combinatory logic, namely that atomic types have no closed inhabitants to feed a variable-based argument:

```
Red (TV n)    t = SNorm t
Red (Arr a b) t = (SNorm t, (x : Term) -> Red a x -> Red b (App t x))
```

3. The three conditions

CR1, reducibility implies normalization, is the first projection, by design. CR2, forward closure under reduction, is a two-line recursion on the type, the arrow case pushing the step under the application with the left congruence. CR3 is the heart: a neutral term all of whose one-step reducts are reducible is itself reducible. At an atomic type it is the accessibility constructor verbatim. At an arrow type its function half must show the application reducible, and the application's own reducts split, by rootNeeds, into head steps, discharged by the hypothesis, and argument steps, discharged by an inner well-founded descent on the argument's normalization, packaged as the mutual helper cr3App. The type index strictly decreases across the mutual cycle and the argument's accessibility strictly decreases within it, and the totality checker accepts both descents.

4. The axioms, computable

The main lemma must show K and S reducible at their axiom types, and here the shape of the proof is pleasingly physical. K itself and S itself are values with no steps at all, so their normalization is the empty accessibility. Their partial applications are values whose only steps are cargo steps, three small lemmas. Their full applications, K x y and S x y z, are neutral, so CR3 applies, and the reducts enumerate: exactly one redex branch each, and cargo branches. The redex branch of K is one word, the hypothesis on x. The redex branch of S is the whole computational content of the distributor in a single line, the reducibilities of x, y, z applied to one another as the axiom dictates. The cargo branches recurse by well-founded descent on the cargos' normalization, delivered as top-level mutual helpers (redKxyGo, redSxyzGo) rather than case expressions, for reasons section 6 records. The main induction over derivations is then three clauses, the modus ponens case a single application, and:

```
strongNorm : {a : Ty} -> {t : Term} -> Deriv a t -> SNorm t
```

5. What it closes, and one corollary in prose

L1 is closed: the coin's dynamics are now fully certified, preservation (the earlier paper) and termination (this one), which together say that a proof, run as a program, remains a proof at every step and stops. One corollary follows by reading this paper next to the reader campaign, and it is worth a paragraph though it is not compiled as a single named theorem. Stage two's engine takes real steps (simSound) and stops only at normal forms (simNormal). Strong normalization says a typed term admits no infinite step sequence. Therefore the engine, pointed at the code of any typed program, halts in finitely many steps at a normal form: the reader always finishes reading a proof. The garden's fence, its price, and its payoff are now all this program's own.

6. Mechanization notes, continued

Three notes for the series' running ledger. First, the parser: a documentation comment may not precede a mutual block; it belongs to the first declaration inside. Second, extending part R's lesson: case expressions inside lambdas neither refine dependent indices reliably nor accept impossible alternatives, so the descent continuations were lifted to top-level mutual helpers, where dependent matching and coverage behave; the pattern, views out, helpers in, now has three confirmations in

this module. Third, scale: at 2,254 lines the module's executable build exceeds this environment's memory, while the checker passes in full; per the series' standing methods statement, idris2 —check is the entire verification, and nothing in this paper depends on running the binary.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 2,254 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification. The executable inventory build is omitted at this size for the environmental reason stated in section 6.

Appendix B: The normalization section (module part S)

```
--------------------------------------------------------------------
-- Tier 2, part S: every typed song ends (L1, second half)
--------------------------------------------------------------------

||| strong normalization as accessibility under Step
data SNorm : Term -> Type where
  MkSN : {t : Term} ->
         ((u : Term) -> Step t u -> SNorm u) -> SNorm t

snStep : {t, u : Term} -> SNorm t -> Step t u -> SNorm u
snStep (MkSN f) s = f u s

||| the calculus's values: under-applied heads, one shape each
data Val : Term -> Type where
  VK  : Val CK
  VS  : Val CS
  VK1 : Val (App CK x)
  VS1 : Val (App CS x)
  VS2 : Val (App (App CS x) y)

||| neutral: not a value; applying a neutral head creates no root redex
Neutral : Term -> Type
Neutral t = Val t -> Void

||| steps of an application with a neutral head happen in the parts
rootNeeds : {t, w, u : Term} -> Neutral t -> Step (App t w) u ->
  Either (t' : Term ** (Step t t', u = App t' w))
         (w' : Term ** (Step w w', u = App t w'))
rootNeeds nt (RedK {a} {b})     = absurd (nt VK1)
rootNeeds nt (RedS {a} {b} {c}) = absurd (nt VS2)
rootNeeds nt (CtxL {l' = t'} s) = Left (t' ** (s, Refl))
rootNeeds nt (CtxR {r' = w'} s) = Right (w' ** (s, Refl))

||| a neutral head keeps its application neutral
neuApp : Neutral t -> Neutral (App t w)
neuApp nt VK1 = nt VK
neuApp nt VS1 = nt VS
neuApp nt VS2 = nt VS1

neuKxy : Neutral (App (App CK x) y)
neuKxy VS2 impossible

neuSxyz : Neutral (App (App (App CS x) y) z)
neuSxyz v impossible
```

```
||| reducibility, with strong normalization carried at every arrow
Red : Ty -> Term -> Type
Red (TV n)    t = SNorm t
Red (Arr a b) t =
  (SNorm t, (x : Term) -> Red a x -> Red b (App t x))

cr1 : {a : Ty} -> Red a t -> SNorm t
cr1 {a = TV n}    r = r
cr1 {a = Arr a b} r = fst r

cr2 : {a : Ty} -> {t, u : Term} -> Red a t -> Step t u -> Red a u
cr2 {a = TV n}    r s = snStep r s
cr2 {a = Arr a b} r s =
  (snStep (fst r) s, \x, rx => cr2 {a = b} (snd r x rx) (CtxL s))

mutual
  cr3 : {a : Ty} -> {t : Term} -> Neutral t ->
    ((u : Term) -> Step t u -> Red a u) -> Red a t
  cr3 {a = TV n}    nt h = MkSN h
  cr3 {a = Arr a b} nt h =
    ( MkSN (\u, s => cr1 {a = Arr a b} (h u s))
    , \w, rw => cr3App nt h w (cr1 {a = a} rw) rw )

  cr3App : {a, b : Ty} -> {t : Term} -> Neutral t ->
    ((u : Term) -> Step t u -> Red (Arr a b) u) ->
    (w : Term) -> SNorm w -> Red a w -> Red b (App t w)
  cr3App nt h w (MkSN sw) rw =
    cr3 (neuApp nt) $ \u, s =>
      case rootNeeds nt s of
        Left (t' ** (st, eq)) =>
          rewrite eq in snd (h t' st) w rw
        Right (w' ** (sw', eq)) =>
          rewrite eq in
          cr3App nt h w' (sw w' sw') (cr2 {a = a} rw sw')

||| K applied once is strongly normalizing when its cargo is
snK1 : {x : Term} -> SNorm x -> SNorm (App CK x)
snK1 (MkSN f) = MkSN $ \u, s =>
  case s of
    CtxL s' => absurd (noStepCK s')
    CtxR {r' = x'} s' => snK1 (f x' s')

mutual
  ||| the K axiom, computable: by descent on both cargos
  redKxy : {a, b : Ty} -> (x, y : Term) ->
    SNorm x -> SNorm y -> Red a x -> Red b y ->
    Red a (App (App CK x) y)
  redKxy x y snx sny rx ry =
    cr3 neuKxy (redKxyGo x y snx sny rx ry)

  redKxyGo : {a, b : Ty} -> (x, y : Term) ->
    SNorm x -> SNorm y -> Red a x -> Red b y ->
    (u : Term) -> Step (App (App CK x) y) u -> Red a u
  redKxyGo x y snx sny rx ry _ RedK = rx
```

```
  redKxyGo x y snx sny rx ry _ (CtxL (CtxL s')) =
    absurd (noStepCK s')
  redKxyGo x y (MkSN sx) sny rx ry _ (CtxL (CtxR {r' = x'} s')) =
    redKxy x' y (sx x' s') sny (cr2 {a = a} rx s') ry
  redKxyGo x y snx (MkSN sy) rx ry _ (CtxR {r' = y'} s') =
    redKxy x y' snx (sy y' s') rx (cr2 {a = b} ry s')

redKAx : {a, b : Ty} -> Red (Arr a (Arr b a)) CK
redKAx =
  ( MkSN (\u, s => absurd (noStepCK s))
  , \x, rx =>
      ( snK1 (cr1 {a = a} rx)
      , \y, ry => redKxy x y (cr1 {a = a} rx) (cr1 {a = b} ry) rx ry ) )

||| S applied once and twice, strongly normalizing on cargo
snS1 : {x : Term} -> SNorm x -> SNorm (App CS x)
snS1 (MkSN f) = MkSN $ \u, s =>
  case s of
    CtxL s' => absurd (noStepCS s')
    CtxR {r' = x'} s' => snS1 (f x' s')

snS2 : {x, y : Term} -> SNorm x -> SNorm y -> SNorm (App (App CS x) y)
snS2 (MkSN fx) (MkSN fy) = MkSN $ \u, s =>
  case s of
    CtxL (CtxL s') => absurd (noStepCS s')
    CtxL (CtxR {r' = x'} s') => snS2 (fx x' s') (MkSN fy)
    CtxR {r' = y'} s' => snS2 (MkSN fx) (fy y' s')

mutual
  ||| the S axiom, computable: by descent on all three cargos
  redSxyz : {a, b, c : Ty} -> (x, y, z : Term) ->
    SNorm x -> SNorm y -> SNorm z ->
    Red (Arr a (Arr b c)) x -> Red (Arr a b) y -> Red a z ->
    Red c (App (App (App CS x) y) z)
  redSxyz x y z snx sny snz rx ry rz =
    cr3 neuSxyz (redSxyzGo x y z snx sny snz rx ry rz)

  redSxyzGo : {a, b, c : Ty} -> (x, y, z : Term) ->
    SNorm x -> SNorm y -> SNorm z ->
    Red (Arr a (Arr b c)) x -> Red (Arr a b) y -> Red a z ->
    (u : Term) -> Step (App (App (App CS x) y) z) u -> Red c u
  redSxyzGo x y z snx sny snz rx ry rz _ RedS =
    snd (snd rx z rz) (App y z) (snd ry z rz)
  redSxyzGo x y z snx sny snz rx ry rz _ (CtxL (CtxL (CtxL s'))) =
    absurd (noStepCS s')
  redSxyzGo x y z (MkSN sx) sny snz rx ry rz _
    (CtxL (CtxL (CtxR {r' = x'} s'))) =
    redSxyz x' y z (sx x' s') sny snz
      (cr2 {a = Arr a (Arr b c)} rx s') ry rz
  redSxyzGo x y z snx (MkSN sy) snz rx ry rz _
    (CtxL (CtxR {r' = y'} s')) =
    redSxyz x y' z snx (sy y' s') snz
      rx (cr2 {a = Arr a b} ry s') rz
  redSxyzGo x y z snx sny (MkSN sz) rx ry rz _
    (CtxR {r' = z'} s') =
```

```
    redSxyz x y z' snx sny (sz z' s')
      rx ry (cr2 {a = a} rz s')

redSAx : {a, b, c : Ty} ->
  Red (Arr (Arr a (Arr b c)) (Arr (Arr a b) (Arr a c))) CS
redSAx =
  ( MkSN (\u, s => absurd (noStepCS s))
  , \x, rx =>
      ( snS1 (cr1 {a = Arr a (Arr b c)} rx)
      , \y, ry =>
          ( snS2 (cr1 {a = Arr a (Arr b c)} rx) (cr1 {a = Arr a b} ry)
          , \z, rz =>
              redSxyz x y z (cr1 {a = Arr a (Arr b c)} rx)
                (cr1 {a = Arr a b} ry) (cr1 {a = a} rz) rx ry rz ) ) )

||| every derivation is computable
redSound : {a : Ty} -> {t : Term} -> Deriv a t -> Red a t
redSound AxK        = redKAx
redSound AxS        = redSAx
redSound (MP {x = x0} df dx) = snd (redSound df) x0 (redSound dx)

||| every typed song ends: strong normalization for the coin
strongNorm : {a : Ty} -> {t : Term} -> Deriv a t -> SNorm t
strongNorm d = cr1 {a = a} (redSound d)
```

References

1. W. W. Tait, ‘Intensional interpretations of functionals of finite type I’, Journal of Symbolic Logic 32 (1967), 198–212.
2. J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*, Cambridge University Press, 1989, chapter 6, for the three conditions.
3. J. R. Hindley, *Basic Simple Type Theory*, Cambridge University Press, 1997.
4. Project secret3, ‘A Verified Coin: Type Preservation for the Two-Move Hilbert System’, working paper v0.1, August 11, 2026.
5. Project secret3, ‘The Reader’s Engine’, working paper v0.1, August 11, 2026, for the corollary of section 5.